

Course 50.050/50.550

Advanced Algorithms

Week 5 – Cohort Class L05.03



Outline of Cohort Class

- ▶ Weakly connected, strongly connected
- ▶ Using DFS to compute connected components
- ▶ Class activity: Rumors in social networks
- ▶ Kosaraju's algorithm
- ▶ Class discussion: Envy-free cake division
- ▶ Sperner colorings, Sperner's lemma



Recap: Connectedness and connected components

Definition: Let $G = (V, E, \phi)$ be an **undirected** graph. If u and v are connected for all possible vertices u, v in V , then we say that G is **connected**, otherwise we say that G is **disconnected**.

- ▶ A **connected component** of G is a connected vertex-induced subgraph $G[U]$ of G for some subset $U \subseteq V$, such that $G[U']$ is disconnected for all $U \subsetneq U' \subseteq V$ (if any).

Example: An undirected graph with 4 connected components.



- ▶ **TFAE** is an abbreviation for “the follow are equivalent”.
 - ▶ For the list of statements that follow after TFAE, if any one statement is true, then all statements are true. If any one statement is false, then all statements are false.

Theorem: Let $G = (V, E, \phi)$ be an **undirected** graph. TFAE:

- ▶ G is connected.
- ▶ For all vertices u, v in G , there exists a walk from u to v .
- ▶ For all vertices u, v in G , there exists a simple path from u to v .
- ▶ For all vertices u, v in G , there exists a trail from u to v .



Weakly and strongly connected components

Definition: Let $G = (V, E, \phi)$ be a **directed** graph.

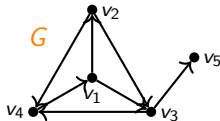
- ▶ We say that G is **weakly connected** if its underlying undirected graph is connected.
- ▶ We say that G is **strongly connected** if for all vertices u, v in V , there is a walk from u to v , and there is a walk from v to u .

Warning: For a directed graph G , it is ambiguous to say G is “disconnected”.

- ▶ Is G not weakly connected? Or is G not strongly connected?

Warning: For a directed graph G , it is ambiguous to say G is “connected”.

Example: Graph G depicted on the right is weakly connected, but not strongly connected.



- ▶ There is no (directed) walk from v_5 to v_3 .

Definition: Let $G = (V, E, \phi)$ be a **directed** graph.

- ▶ A **weakly connected component** of G is a weakly connected vertex-induced subgraph $G[U]$ of G for some subset $U \subseteq V$, such that $G[U']$ is not weakly connected for all $U \subsetneq U' \subseteq V$ (if any).
- ▶ A **strongly connected component** of G is a strongly connected vertex-induced subgraph $G[U]$ of G for some subset $U \subseteq V$, such that $G[U']$ is not strongly connected for all $U \subsetneq U' \subseteq V$ (if any).



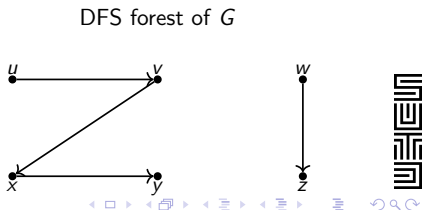
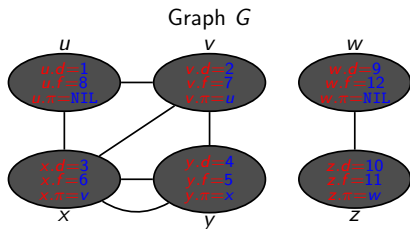
Recap: DFS forest of a graph

Recall: After traversing a graph G via depth-first search (DFS), we can construct a **DFS forest** G' , which is a directed forest.

- ▶ Whether G is undirected or directed, its DFS forest G' is **directed**.
 - ▶ **Note:** $V(G) = V(G')$, i.e. the DFS forest has the same vertex set.
- ▶ After running DFS, each $x \in V(G)$ has an attribute value $x.\pi$ that is either NIL, or a vertex that represents the **predecessor** of x .
 - ▶ **Intuition:** x was “discovered by” $x.\pi$.
 - ▶ **Intuition:** $x.\pi$ is the parent of x in the DFS forest.

Definition: Let G be a graph, and suppose we have run DFS on G . The **DFS forest** (or **predecessor graph**) of G is a simple directed graph $G' = (V', E')$ where $V' = V$, and $E' = \{(x.\pi, x) | x \in G.V, x.\pi \neq \text{NIL}\}$.

DFS Example: Within each depth level, traverse vertices in alphabetical order.

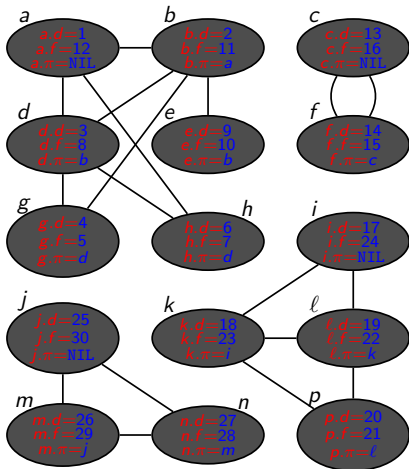


Using DFS to compute connected components

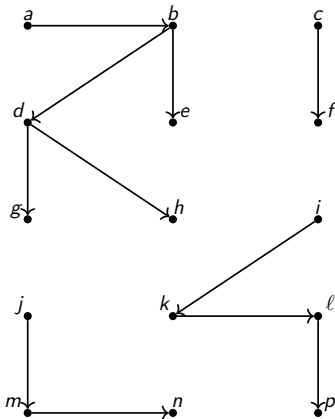
Idea: $\left\{ \begin{array}{l} \text{Vertex sets of the} \\ \text{connected components} \\ \text{of an undirected graph } G \end{array} \right\} = \left\{ \begin{array}{l} \text{Vertex sets of the} \\ \text{weakly connected components} \\ \text{of the DFS forest of } G \end{array} \right\}.$

Example: Within each depth level, traverse vertices in alphabetical order.

Graph G



DFS forest of G



DFS with count of number of connected components

function DFS-COUNT(G)

Require: $G = (G.V, G.E, G.phi)$
is a graph.

Require: $G.Inc$ is the incidence list of G .

```
1: for each vertex  $u$  in  $G.V$  do
2:    $u.color \leftarrow$  'WHITE'
3:    $u.\pi \leftarrow$  NIL
4:  $time \leftarrow 0$ 
5:  $count \leftarrow 0$ 
6: for each vertex  $u$  in  $G.V$  do
7:   if  $u.color =$  'WHITE' then
8:      $count \leftarrow count + 1$ 
9:     DFS-VISIT( $G, u$ )
10: return  $count$ 
```

function DFS-VISIT(G, u)

Require: $G = (G.V, G.E, G.phi)$
is a graph.

Require: $G.Inc$ is the incidence list of G .

Require: u is a vertex in $G.V$.

```
1:  $time \leftarrow time + 1$ 
2:  $u.d \leftarrow time$ 
3:  $u.color \leftarrow$  'GRAY'
4:  $e \leftarrow G.Inc[u].head$ 
5: while  $e \neq$  NIL do
6:    $v \leftarrow$  OTHERENDPOINT( $e, u$ )
7:   if  $v.color =$  'WHITE' then
8:      $v.\pi \leftarrow u$ 
9:     DFS-VISIT( $G, v$ )
10:   $e \leftarrow e.next$ 
11:  $u.color \leftarrow$  'BLACK'
12:  $time \leftarrow time + 1$ 
13:  $u.f \leftarrow time$ 
```

DFS with array of connected components

function DFS-ARRAY(G)

Require: $G = (G.V, G.E, G.phi)$
is a graph.

Require: $G.Inc$ is the incidence list
of G .

```
1: for each vertex  $u$  in  $G.V$  do
2:    $u.color \leftarrow$  'WHITE'
3:    $u.\pi \leftarrow$  NIL
4:  $time \leftarrow 0$ 
5: Initialize empty array  $CC$ 
6: for each vertex  $u$  in  $G.V$  do
7:   if  $u.color =$  'WHITE' then
8:     Initialize empty array  $A$ 
9:     DFS-VISIT-TRACK( $G, u, A$ )
10:    Append array  $CC$  with entry  $A$ 
11: return  $CC$  (i.e. an array of arrays)
```

function DFS-VISIT-TRACK(G, u, A)

Require: $G = (G.V, G.E, G.phi)$
is a graph.

Require: $G.Inc$ is the incidence list of G .

Require: u is a vertex in $G.V$.

Require: A is an array with entries
in $G.V$.

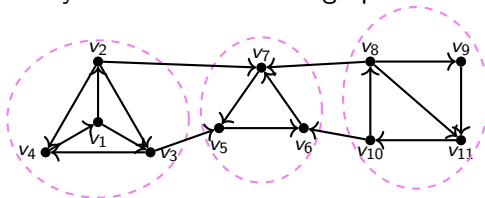
```
1:  $time \leftarrow time + 1$ 
2:  $u.d \leftarrow time$ 
3:  $u.color \leftarrow$  'GRAY'
4:  $e \leftarrow G.Inc[u].head$ 
5: while  $e \neq$  NIL do
6:    $v \leftarrow$  OTHERENDPOINT( $e, u$ )
7:   if  $v.color =$  'WHITE' then
8:      $v.\pi \leftarrow u$ 
9:     DFS-VISIT-TRACK( $G, v, A$ )
10:   $e \leftarrow e.next$ 
11:  $u.color \leftarrow$  'BLACK'
12:  $time \leftarrow time + 1$ 
13:  $u.f \leftarrow time$ 
14:  $A \leftarrow$  array obtained by prepending  
   $A$  with entry  $u$ 
```



More on strongly connected components

Useful abbreviation: SCC for “strongly connected component”.

Example: A weakly connected directed graph with 3 SCCs.



Theorem: Let $G = (V, E, \phi)$ be a **directed** graph.

- ▶ For any two distinct vertices u, v in the same SCC of G , there exists a closed walk containing u and v .
- ▶ Let u, v be vertices in different SCCs of G . If there is a walk from u to v , then there does NOT exist any walk from v to u .

Proof Idea: This follows from the definitions. (*Can you see why?*)

Definition: Let $G = (V, E, \phi)$ be a **directed** graph, and let $e \in E$ be an edge. If the number of SCCs in the deletion $G \setminus e$ is strictly larger than the number of SCCs in G , then e is called a **strong bridge** (or **strong cut edge**) of G .

After our successful bird conservation week,
there has been some rumors going around.
The rumors are all over TweetTweet!



Rumors in social networks

In John's world, everyone is friends with everyone, and everyone is on this social network called TweetTweet.

- ▶ Any TweetTweet user can choose to “follow” any other TweetTweet user.
 - ▶ Among John's friends, not everyone follows everyone on TweetTweet.
- ▶ Any TweetTweet user can choose to re-post a TweetTweet post of a TweetTweet user he/she/it is following.
 - ▶ John's friends have the habit of seeing every TweetTweet post of every TweetTweet user they follow.
 - ▶ John's friends have the habit of re-posting every TweetTweet post!

Definition: John defines a **TweetCluster** to be a group of friends such that for any two friends A and B in the group, A would be able to see any TweetTweet posts from B , and vice versa.

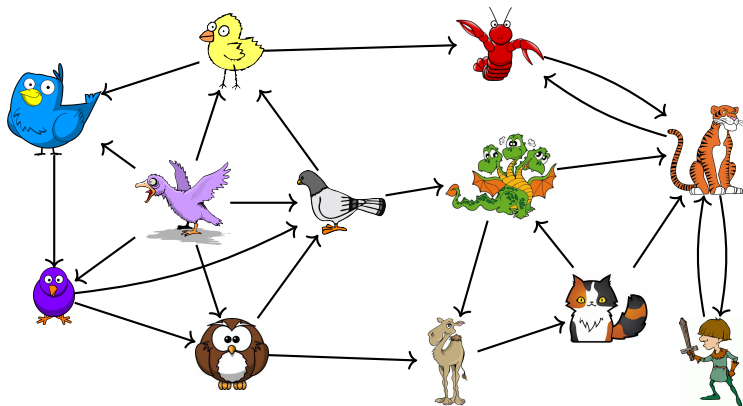


Can you help me find all TweetClusters?
I would like to know who is in which
TweetCluster.



Class Activity: TweetClusters (15 mins)

Here are the TweetTweet “follows”. $x \bullet \rightarrow y$ means x “follows” y .



Questions:

1. Find all TweetClusters. How many TweetClusters are there?
 - Can you find all TweetClusters in a systematic way?
2. Is it possible to insert just one more “follow” edge $x \bullet \rightarrow y$ so that there is only one TweetCluster?
 - Find all possible edges that could be this single “follow” edge.



Latest finishing times for SCCs

Definition: Suppose we have run DFS on a graph G . Then for any $U \subseteq V(G)$, let $d_G(U) := \min\{u.d \mid u \in U\}$, $f_G(U) := \max\{u.f \mid u \in U\}$.

► $d_G(U)$ = **earliest discovery time**. $f_G(U)$ = **latest finishing time**.

Theorem: Let $G = (V, E, \phi)$ be a directed graph. Suppose there is an edge $e \in E$ with $\phi(e) = (u, u')$, where there exists disjoint subsets $U, U' \subseteq V$, containing u, u' respectively, such that $G[U]$ and $G[U']$ are distinct SCCs of G . Then $f_G(U) > f_G(U')$.

Proof: Consider the following two cases.

[Case 1]: $d_G(U) < d_G(U')$. Let x be earliest discovered vertex in U .

- At time $x.d$, right before x is colored 'GRAY', there is a walk from x to y for any vertex $y \in U'$, such that all vertices in this walk are 'WHITE'.
 - Walk from x to u , then traverse edge e , then walk from u' to w .
- Since DFS always traverses deeper whenever possible, all vertices y in U' will become 'BLACK' before x is colored 'BLACK'. Thus, $f_G(U) > f_G(U')$.

[Case 2]: $d_G(U) > d_G(U')$. Let y' be earliest discovered vertex in U' .

- **Note:** For any vertex $x' \in U$, there is no walk from y' to x' .
 - Can you see why this is true, using edge e ?
- All vertices in U' will be colored 'BLACK' before x' is colored 'BLACK', thus $f_G(U) > f_G(U')$.

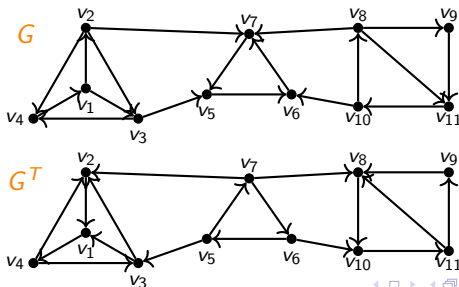


Transpose graphs

Definition: Let $G = (V, E, \phi)$ be a **directed** graph. The **transpose graph** of G is the graph $G' = (V', E', \phi')$ where $V' = V$, $E' = E$, such that for every edge $e \in E'$, if $\phi(e) = (u, v)$, then $\phi'(e) = (v, u)$.

- ▶ **Intuition:** To get this transpose graph, we **reverse** the directions of all directed edges.
- ▶ **Note:** For a simple directed graph G with adjacency matrix A , the adjacency matrix of the transpose graph of G is the transpose of A .
- ▶ Also called **reverse graph** or **reversal graph** or **converse graph**.
- ▶ **No standardized notation!** Possible notation: G^T, G^R .

Example: A directed graph G and the transpose graph G^T of G .



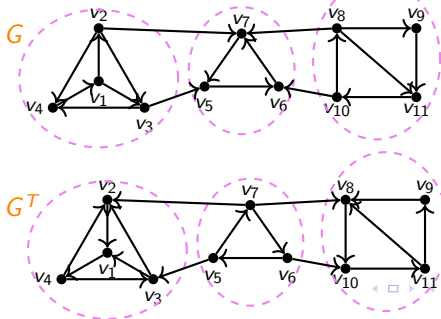
Intuition for Kosaraju's algorithm

Kosaraju's algorithm: To compute the SCCs in a **directed** graph G .

Key Idea 1: Consider the **transpose graph** G^T of G .

- ▶ For any two vertices u, v in the same SCC of G , if there is a walk from u to v in G , then there is also a walk from u to v in G^T .
- ▶ For any two vertices u, v in two distinct SCCs of G , if there is a walk from u to v in G , then there are no walks from u to v in G^T .
- ▶ **Important Consequence:** The vertex sets of the SCCs of G and G^T are exactly the same!
 - ▶ **Note:** An SCC is a vertex-induced subgraph, and $V(G) = V(G^T)$.

Example:



Intuition for Kosaraju's algorithm (continued)

Assume we have run DFS on G , and suppose that $U, U' \subseteq V(G)$ such that $G[U], G[U']$ are distinct SCCs in G , and such that $f_G(U) > f_G(U')$.

Key Idea 2: Since $f_G(U) > f_G(U')$, the theorem on slide 13 tells us that if there is an edge with the two endpoints in U and U' , then this edge must be from a vertex in U to a vertex in U' .

- ▶ Hence, in the transpose graph G^T , there are NO edges from any vertex in U to any vertex in U' .
- ▶ **Important Consequence:** If we run DFS on G^T and visit some vertex in U first, then we must visit all vertices in U first, before visiting any vertex in U' .

Key Idea 3: After running DFS on G , we run DFS again on G^T , but based on the finishing times from our first DFS as our order for $V(G^T)$.

- ▶ From Key Idea 2, this would mean that for each SCC, we would visit all vertices in that SCC, before we visit another SCC.
- ▶ After this second DFS (on G^T),

$$\left\{ \begin{array}{c} \text{Vertex sets of the} \\ \text{strongly connected components} \\ \text{of the transpose graph } G^T \end{array} \right\} = \left\{ \begin{array}{c} \text{Vertex sets of the} \\ \text{weakly connected components} \\ \text{of the DFS forest of } G^T \end{array} \right\}.$$



Kosaraju's algorithm

- 3 main steps in Kosaraju's algorithm:** Given a directed graph G ,
1. Run DFS on G . Store the vertices of G in a new array A in descending order of their finishing times for this first DFS.
 - ▶ The vertex with the largest finishing time is the first vertex in A .
 2. Construct the transpose graph G^T of G .
 3. Set $G^T.V = A$, and run DFS on G^T .
 - ▶ The weakly connected components of the DFS forest of G^T correspond to the SCCs of G .

Note: Kosaraju's algorithm has **two** runs of DFS.

- ▶ Each of the 3 steps has time complexity $O(|V(G)| + |E(G)|)$, hence the overall time complexity $O(|V(G)| + |E(G)|)$.

Subroutines for Kosaraju's algorithm

function DFS-TRACK(G)

Require: $G = (G.V, G.E, G.phi)$ is a graph.

Require: $G.Inc$ is the incidence list of G .

1: **for** each vertex u in $G.V$ **do**

2: $u.color \leftarrow \text{'WHITE'}$

3: $u.\pi \leftarrow \text{NIL}$

4: $time \leftarrow 0$

5: **Initialize empty array** A

6: **for** each vertex u in $G.V$ **do**

7: **if** $u.color = \text{'WHITE'}$ **then**

8: DFS-VISIT-TRACK(G, u, A)

9: **return** A

function DFS-VISIT-TRACK(G, u, A)

Require: $G = (G.V, G.E, G.phi)$ is a graph.

Require: $G.Inc$ is the incidence list of G .

Require: u is a vertex in $G.V$.

Require: A is an array with entries in $G.V$.

1: $time \leftarrow time + 1$

2: $u.d \leftarrow time$

3: $u.color \leftarrow \text{'GRAY'}$

4: $e \leftarrow G.Inc[u].head$

5: **while** $e \neq \text{NIL}$ **do**

6: $v \leftarrow \text{OTHERENDPOINT}(e, u)$

7: **if** $v.color = \text{'WHITE'}$ **then**

8: $v.\pi \leftarrow u$

9: DFS-VISIT-TRACK(G, v, A)

10: $e \leftarrow e.next$

11: $u.color \leftarrow \text{'BLACK'}$

12: $time \leftarrow time + 1$

13: $u.f \leftarrow time$

14: $A \leftarrow$ array obtained by prepending A with entry u



Pseudocode for Kosaraju's algorithm

function KOSARAJU(G)

Require: $G = (G.V, G.E, G.phi)$ is a directed graph.

Require: $G.Inc$ is the incidence list of G .

- 1: $A \leftarrow \text{DFS-TRACK}(G)$
- 2: $\text{TRANSPOSEGRAPH}(G)$
- 3: $G.V \leftarrow A$
- 4: $\text{SCC} \leftarrow \text{DFS-ARRAY}(G)$
- 5: **return** SCC

function TRANSPOSEGRAPH(G)

Require: $G = (G.V, G.E, G.phi)$ is a directed graph.

Require: $G.Inc$ is the incidence list of G .

- 1: Initialize array L indexed by $G.V$, whose entries are empty linked lists
- 2: **for** each edge e in $G.E$ **do**
- 3: $[u, v] \leftarrow G.phi[e]$
- 4: $G.phi[e] \leftarrow [v, u]$
- 5: $\text{LIST-INSERT}(L[v], e)$
- 6: $G.Inc \leftarrow L$

Task of the day: Run Kosaraju's algorithm on the TweetTweet graph.

- Check that you get the same SCCs.

Some applications of Kosaraju's algorithm

- ▶ Model spread of disease (e.g. COVID-19) in communities
 - ▶ Vertices represent people, SCCs represent groups of people susceptible to infection within group.
- ▶ Model spread of information over social networks
 - ▶ Vertices represent people, SCCs represent groups of people who can “influence” each other.
- ▶ Supply chain management
 - ▶ Vertices represent countries/companies, SCCs represent groups resilient to supply chain disruptions.
- ▶ Targeted advertising
 - ▶ Vertices represent people, SCCs represent groups of people with common interests
- ▶ Internet traffic management
 - ▶ Vertices represent devices, SCCs represent groups of devices with good Internet coverage

I haven't told you the rumors going around!
First of all, three of our participants from last
week's bird conservation efforts have been
selected for their outstanding performance.



For their outstanding performance, each of them
will be rewarded with a share of a large fruit cake!
Rumors have been going around that they will
not be able to agree on how to divide the cake.



Cake division

Three participants from last week's bird conservation efforts have been selected to share a prize cake.



- ▶ The rectangular prize cake has lots of fruit toppings, with different parts of the cake having different mixtures of fruits.
- ▶ For simplicity, the cake will be cut into three pieces of sizes α, β, γ , from left to right, where $\alpha + \beta + \gamma = 1$.
 - ▶ First piece is leftmost α portion of the cake; third piece is rightmost γ portion of the cake; second piece is middle β portion of the cake.
 - ▶ **Big headache:** The three winners have their own different views on what a “best cake” means, and they want the “best cake”!
 - ▶ e.g. more strawberries is “better”, less cherries is “better”, etc.
 - ▶ For any given cake division, it is possible that different winners have different opinions on which is the “best” piece of cake.



Class Discussion: Envy-free cake division (15 mins)



Discussion: Is it possible to systematically find some 3-tuple (α, β, γ) for the cake division, and give **different** cake pieces to the 3 winners, so that all 3 winners unanimously agree that no other cake piece is (in their own opinion) **strictly better** than the piece they were given?

- ▶ You can query each of them any (α, β, γ) (for cake division) to find out which piece they think is the best piece.
 - ▶ Each of them has their own individual “utility function”, assigning their own perceived values to the 3 pieces of cake for any given cake division. “Envy-free” means that for each winner X , the cake piece given to X has the maximum perceived value (as perceived by X) among the 3 pieces.

Assumptions:

- ▶ All of them prefer cake over no cake.
- ▶ Each winner's utility function varies continuously with (α, β, γ) .
 - ▶ If winner X prefers the same cake for a convergent sequence $(\alpha_1, \beta_1, \gamma_1), (\alpha_2, \beta_2, \gamma_2), \dots$, then X will also prefer the same cake in the limit.



Summary

- ▶ Weakly connected, strongly connected
- ▶ Using DFS to compute connected components
- ▶ Class activity: Rumors in social networks
- ▶ Kosaraju's algorithm
- ▶ Class discussion: Envy-free cake division
- ▶ Sperner colorings, Sperner's lemma

